UNIVERSITI SAINS MALAYSIA

School of Electrical and Electronics Engineering
Universiti Sains Malaysia, Engineering Campus

SEMESTER 1, 2026/2027

EEM 441

Control & Instrumentation Laboratory

Experiment 8

TEMPERATURE MEASUREMENT AND CONTROL

Date : _____

Group No. : _____

Group Members : _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

: _____ Matrix # _____

Experiment 8

Temperature Control Using Raspberry Pi

Objectives

1. To interface a temperature sensor and a heating element with a Raspberry Pi.
2. To implement a temperature control algorithm to maintain a target temperature.
3. To log temperature data and analyse system performance.

Equipment / Software Required

- Raspberry Pi 4B
- NTC thermistor (or other temperature sensor) and trainer board with indicator LED
- Analog-to-digital converter, **ADC0804**
- Operational amplifier, LM741
- Relay
- Resistors, capacitors (according to your design)
- NPN transistor, 2N3904
- LEDs
- Power supply and connecting wires

Introduction

In most industrial processes, there are possibilities when the temperature needs to be regulated automatically. For example, the temperature of fluid inside a reactor has to be controlled within a certain range in order to get a stable operation. In logic control, the temperature is controlled by switching the LED on or off when a certain critical value is reached, whereas fuzzy control needs a true definition of “hot” before the control operation.

In this open-ended lab, students will design and implement a system to perform temperature control using a Raspberry Pi. The goal is to create a complete system that captures the temperature sensor reading, processes the information, displays the temperature, and turns an LED on or off based on the temperature.

An overall block diagram for the measurement system is shown in Figure 8.1.

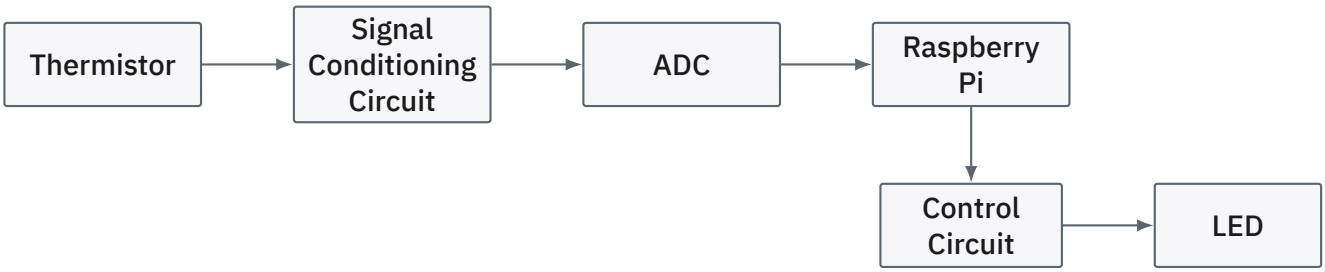


Figure 8.1: Block diagram of the temperature control system.

Procedure

Activity 1 — Signal-conditioning circuit design

Objective: design and build a circuit that converts the thermistor's resistance to a voltage suitable for the ADC0804 input.

1. Obtain the resistance–temperature characteristic for the NTC thermistor from its datasheet, and determine the resistance range corresponding to the temperature range you intend to measure and control.
2. Design a signal-conditioning circuit that converts the thermistor's resistance to a voltage. Calculate your component values and justify your design choices, including the expected output voltage range over your chosen temperature range.
3. Include a voltage-follower (buffer) stage, based on the LM741, to eliminate loading effects between the conditioning circuit and the ADC0804 input.
4. Build the circuit and verify its output voltage against a reference temperature measurement at two or more points, and check linearity/consistency with your design calculations.

Activity 2 — Configure and test the ADC

Objective: build and verify the ADC0804 circuit in isolation, before integrating it with the Raspberry Pi.

1. Configure and construct the ADC0804 in free-running mode, with its input connected to a potentiometer and its outputs connected to LEDs. Test that the LED pattern changes correctly as the potentiometer is adjusted.
2. Configure and construct the relay/BJT driver stage that will switch the LED (or indicator LED) on and off from a single digital output level. Test the stage directly (e.g. with a switch or a logic-level source) before connecting it to the Raspberry Pi, confirming the ON/OFF output you intend to use.

Activity 3 — Interfacing to the Raspberry Pi

Objective: connect the ADC0804 to the Raspberry Pi via a bidirectional level shifter, and connect a GPIO output to the LED control circuit.

1. Connect the output of your signal-conditioning circuit (Activity 1) to the V_{in} input of the ADC0804.
2. Wire the ADC0804's 5 V-side signals (D0–D7, $\overline{\text{INTR}}$, $\overline{\text{CS}}$, $\overline{\text{RD}}$, $\overline{\text{WR}}$) to the high-voltage (HV) side of a bidirectional level-shifter module, and the corresponding Raspberry Pi GPIO pins to the low-voltage (LV) side.
3. Connect a Raspberry Pi GPIO output pin to your LED control circuit (relay driven via the NPN 2N3904 transistor), which in turn drives the trainer's LED/indicator LED.
4. Verify both the input and output wiring with a simple test script before writing the full control program: confirm a static ADC reading is read back correctly, and confirm the LED/LED can be toggled on and off by setting the GPIO output pin.

Activity 4 — Temperature monitoring and LED control software

Objective: implement software that displays the measured temperature and controls the LED according to a threshold.

1. In Python or C/C++, write a program that:
   - periodically triggers an ADC0804 conversion and reads back the digitised thermistor voltage,
   - converts the reading to a temperature using your circuit's calibration from Activity 1,
   - displays the current temperature in a simple terminal or GUI display, updating in real time, and
   - compares the measured temperature against the threshold and sets the GPIO output level to control the LED.
2. Report and justify your design choice for the signal-conditioning circuit.
3. Set the controlled (threshold) temperature to, for example, $20\,^{\circ}\text{C}$, and record the observed thermostat temperature for 35 minutes using a 1-minute time interval (Table 8.1). Plot temperature against time.
4. Repeat for $23\,^{\circ}\text{C}$ and $25\,^{\circ}\text{C}$. Plot your results.
5. Discuss your findings, including the responsiveness of the LED control and the accuracy of your temperature conversion.

Table 8.1: Temperature and LED-state log.

Threshold temp.	Time (min)	Measured temp.	LED state
	1		
	2		
	3		
	4		
	5		
	6		
	7		
	8		
	9		
	10		
	11		
	12		
	13		
	14		
	15		
	16		
	17		
	18		
	19		
	20		
	21		
	22		
	23		
	24		
	25		
	26		
	27		
	28		
	29		
	30		
	31		
	32		
	33		
	34		
	35		

Verified by

Date:

Report

1. Present your signal-conditioning circuit design (topology, calculations, component values) and its calibration against a reference temperature measurement.
2. Describe your ADC0804 interfacing to the Raspberry Pi, including your level-shifting solution and why it is needed between the ADC0804's 5 V logic and the Raspberry Pi's 3.3 V GPIO.
3. State and justify your chosen LED ON/OFF logic.
4. Present and discuss your temperature-vs-time plots for each threshold tested ($20\,^{\circ}\mathrm{C}$, $23\,^{\circ}\mathrm{C}$, $25\,^{\circ}\mathrm{C}$).

5. Assess the system's accuracy (e.g. $\pm 1\,^{\circ}\mathrm{C}$ margin), response time after a setpoint change, and stability (absence of oscillation/overshoot).
6. What sources of error or lag affect the accuracy and responsiveness of this system (thermistor thermal mass, signal-conditioning bandwidth, ADC conversion time, software polling rate, etc.)?

Deliverables

1. **Code:** a programming script that accurately measures and displays the temperature based on sensor inputs, and controls the LED accordingly.
2. **System setup diagram:** a diagram showing the placement and wiring of sensors to the Raspberry Pi.
3. **Documentation:** a report that explains the setup, the temperature control process, sample readings, discussion, and any challenges encountered.
4. **Demo:** a short presentation or video demonstrating the working system and the collected data.
5. **Video:** a video of your work, showing the Raspberry Pi system measuring the temperature and switching the LED ON and OFF as the temperature crosses your chosen thresholds.

References

- ADC0804 datasheet.
- NTC thermistor datasheet: http://www.farnell.com/datasheets/307623.pdf

Setting up Raspberry Pi for the first time

- How to Set Up a Raspberry Pi for the First Time | Tom's Hardware
- https://core-electronics.com.au/guides/temperature-sensing-with-raspberry_pi/
- https://www.cl.cam.ac.uk/projects/raspberrypi/tutorials/temperature/
- https://www.circuitbasics.com/raspberry-pi-ds18b20-temperature-sensor-tutorial/
- https://www.hackster.io/vinayyn/multiple-ds18b20-temp-sensors-interfacing-with-raspberry-pi-d8a6b0
- https://code.visualstudio.com/docs/setup/raspberry-pi
- https://www.raspberrypi.com/news/coding-on-raspberry-pi-remotely-with-visual-studio-code/
- https://www.youtube.com/watch?v=TZRGzLv57mc
- https://www.youtube.com/watch?v=aCGbQB8K8T8

- https://www.youtube.com/watch?v=R6VJE7WfXt8
- https://youtu.be/aSr_k6-dxfk?si=_KWhLcS3gfyPJoL8

Coding on Raspberry Pi remotely with Visual Studio Code

- https://besomi.com/coding-on-raspberry-pi-remotely-with-visual-studio-code/
- https://github.com/gloveboxes/Raspberry-Pi-with-Visual-Studio-Code-Remote-SSH-and-C-or-C-Development/blob/master/README.md